Studio Nx

백엔드 275 — 트랙 연결 모델

2026-10-19

백엔드 275 — 트랙 연결 모델 (models/track_connection.py)

항목	내용
파일	`studionx/backend/app/models/track_connection.py`
작업	S1-10a 클립·연결·패턴 API: 트랙 간 연결 표 모델 (PRD 02 FR-2197)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 274 `Clip` 모델, 다음 276 `schemas/clip.py` 와 `schemas/connection.py`

1. 하는 일

화면의 Connect to Another 칸이 저장하는 연결 한 건을 담는 SQLAlchemy 모델입니다. 모델은 실제 MySQL 표 `track_connections` 와 1:1 로 맞춰져 있고, `studionx/db/02_phase2_sequencer.sql` 과 `docs/DB.md` §3.2 의 정의와 열 이름이 같습니다.

```
track_connections 한 행 (보내는 트랙마다 최대 하나, DB UNIQUE uq_connections_from)
├ from_track_id       – 보내는 트랙
├ to_track_id         – 받는 트랙
├ target              – PLUGIN(이펙트 입력) 또는 INSTRUMENTS(악기 입력), 기본 PLUGIN
├ to_track_plugin_id  – target 이 PLUGIN 일 때 받는 트랙의 이펙트 한 칸
├ to_port             – 받는 트랙의 입력 번호, 기본 1
├ also_play           – NULL 이면 자동, true/false 면 켜짐, 꺼짐을 직접 정함
└ volume_db, pan, is_solo, created_at
```

보내는 트랙 하나에 연결은 하나뿐입니다. DB 의 `UNIQUE uq_connections_from` 가 이를 막고, 그래서 저장은 새로 만드는 것이 아니라 보낸 값으로 통째로 바꾸는 방식으로 다룹니다. 보내는 트랙이나 받는 트랙을 지우면 DB 가 `ON DELETE CASCADE` 로 연결도 함께 지웁니다.

쓰는 곳은 `connection_service` 의 조회, 통째로 맞추기, 끄기, `api/v1/connections.py`, 그리고 `full_project_service` 가 트랙마다 채우는 `/projects/{id}/full` 의 `connections` 목록입니다.

2. 코드

```
class ConnectionTarget(StrEnum):
    """받는 쪽에서 무엇으로 받을지. PLUGIN 이면 이펙트 입력, INSTRUMENTS 면 악기 입력이다."""

    PLUGIN = "PLUGIN"
    INSTRUMENTS = "INSTRUMENTS"


class TrackConnection(Base):
    """트랙 → 트랙 연결 한 건. 보내는 트랙마다 최대 하나다 (UNIQUE from_track_id)."""

    __tablename__ = "track_connections"

    connection_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
```

```
    from_track_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE")
    )
    to_track_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tracks.track_id", ondelete="CASCADE")
    )
    target: Mapped[ConnectionTarget] = mapped_column(
        Enum(ConnectionTarget), server_default=ConnectionTarget.PLUGIN
    )
    to_track_plugin_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")
    )
    to_port: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("1"))
    also_play: Mapped[bool | None] = mapped_column(Boolean)
    volume_db: Mapped[Decimal] = mapped_column(Numeric(5, 2), server_default=text("0.00"))
    pan: Mapped[Decimal] = mapped_column(Numeric(3, 2), server_default=text("0.00"))
    is_solo: Mapped[bool] = mapped_column(Boolean, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

문법 풀이

식	뜻
`__tablename__ = "track_connections"`	이 모델이 실제 표 `track_connections` 를 담당함을 적습니다
`ForeignKey("tracks.track_id", ondelete="CASCADE")` (`from_track_id` 와 `to_track_id`)	보내는 트랙이나 받는 트랙이 지워지면 연결도 함께 지워집니다
`ForeignKey("track_plugins.track_plugin_id", ondelete="CASCADE")`	이펙트 칸이 지워지면 그 칸을 가리키던 연결도 함께 지워집니다
`Enum(ConnectionTarget), server_default=ConnectionTarget.PLUGIN`	값을 안 보내면 `PLUGIN` 으로 채워집니다
`Mapped[bool | None]`	`also_play` 처럼 세 가지 경우를 담습니다. NULL 은 자동 결정을 뜻합니다
`TINYINT(unsigned=True)`	입력 번호는 0 부터 255 까지의 정수이고, `to_port` 기본값은 1 입니다
`Numeric(5, 2)` 와 `Numeric(3, 2)`	`volume_db` 와 `pan` 을 소수 둘째 자리까지 담습니다
`DATETIME(fsp=3)`	`created_at` 을 밀리초 단위로 적습니다. 이 표에는 `updated_at` 칸이 없습니다

3. 왜 이렇게

선택	이유
UNIQUE 를 SQL 쪽에만 둠	`uq_connections_from` 은 `02_phase2_sequencer.sql` 에 이미 있습니다. 모델에 `__table_args__` 로 다시 쓰면 같은 규칙이 두 곳에서 어긋날 수 있습니다
저장을 PUT 으로 둠	만들기와 고치기를 나누면 두 행을 넣는 경로가 생깁니다. 보낸 값으로 통째로 맞추면 늘 행 하나여서 UNIQUE와 맞습니다 (FR-2197)
양쪽 트랙 FK 모두 CASCADE	받는 트랙을 지우면 연결이 자연스럽게 사라집니다. 화면이 보는 연결이 None 으로 돌아가는 것과 코드 한 줄 없이 일치합니다

선택	이유
`also_play` 를 NULL 허용	NULL 이면 받는 쪽이 소리도 내는지를 자동으로 정하고, 켜짐 또는 꺼짐을 직접 고르면 true/false 가 들어갑니다
`target` 기본값을 PLUGIN 으로	화면에서 Connect 를 켜면 이펙트 입력으로 먼저 연결되는 동작과 같습니다
자기 자신, 서로, 고리 검사를 스키마에 안 씀	두 트랙 사이의 관계는 표 한 줄로는 알 수 없습니다. `connection_service` 가 400 으로 막습니다
`created_at` 만 있고 `updated_at` 없음	DB 표 정의 그대로입니다. 연결을 고쳐도 만든 시각만 남습니다

4. 검증

`tests/test_track_connection_model.py` 네 건이 모두 통과합니다. `test_connection_columns_match_db` 는 칸 이름, 순서, NULL 허용, FK 를 실제 MySQL 표와 대조하고(assert_matches_db), `test_connection_defaults` 는 기본값(`target` 이 `PLUGIN`, `to_port` 가 1, `also_play` 가 NULL)을, `test_deleting_source_track_deletes_connection` 은 보내는 트랙을 지울 때의 CASCADE, `test_deleting_target_track_deletes_connection` 은 받는 트랙을 지울 때의 CASCADE 를 확인합니다. `tests/test_full_project_routes.py` 의 `test_full_includes_clips_and_connections` 와 `tests/test_connection_routes.py` 도 함께 통과합니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 함수 문서(274 부터 276 까지)가 모두 끝난 뒤 한 번에 검토합니다
DB 변경 없음	`track_connections` 표는 이미 로컬 MySQL에 있어 마이그레이션을 따로 실행하지 않습니다